

Today's Agenda

Topics Covered

1. Function Features
 - Inline Functions
 - Default Arguments
 - Function Overloading
2. Class and Object
3. **this** Pointer
4. Member Functions
 - Constructor
 - Inspector
 - Mutator
 - Destructor
 - Facilitator
5. **const** Variable, Function, and Object

Inline Functions

When you do not want to create a stack frame for a function and want to request the compiler to replace the function call with the function body, you can declare the function as **inline**.

- **inline** is a keyword.
- **inline** is a **request** to the compiler, not a command.
- If your function contains complex logic or a loop, the compiler will most likely **decline** the request.

Function Overloading

Defining the same function multiple times with a **different parameter list** (also called a different **signature**) is called function overloading.

A signature must differ in at least one of the following:

1. Number of parameters
2. Data type of parameters
3. Sequence (order) of parameters

The compiler calls the appropriate function using a **mangled name**. The process of generating these internal names is called **name mangling**.

Function overloading is an example of **compile-time polymorphism**.

Default Arguments

When a value is not passed to a function and you want to use a predefined value instead, you can declare that parameter as a **default parameter** by assigning it a value in the function declaration.

Rule: Default values must be assigned **from right to left** only.

Class and Object

Class

- A class is a collection of similar or dissimilar data types grouped together.
- Class members are **private** by default.
- It is a **logical entity** — a template or blueprint.
- A class itself has **no memory**.

Object

- An object is a real **instance** created from a class (blueprint).
- It is a **physical entity**.
- It has a **physical existence** and occupies **memory**.

Syntax

```
// Structure syntax (C-style, still valid in C++)
struct Time {
    // members
};

// Class syntax
class Time {
    // members
};

// In C++, the 'struct' keyword is optional when declaring a variable
struct Time t1; // t1 is a variable of struct Time
Time t1;       // t1 is an object of class Time
```

Size of an object = sum of all **non-static data members**

- Data members get space **inside the object**.
 - Member functions get space in the **code segment** (not inside the object).
-

this Pointer

When a member function is called on an object, the **address of the calling object** (current object) is automatically passed to the function as its first argument. A special pointer, called the **this pointer**, is available inside every member function to hold this address.

- **Syntax:** `ClassName* const this`

- Writing `this` explicitly is **optional** in most cases, but becomes **mandatory** in certain situations (to be covered later).
-

Member Functions

Functions defined inside a class are called **member functions**. There are five types:

1. **Constructor**
 2. **Inspector** (Getter)
 3. **Mutator** (Setter)
 4. **Destructor**
 5. **Facilitator**
-

Constructor

A constructor is a **special member function** of a class.

Why it is special:

1. It has the **same name** as the class.
2. It has **no return type** (not even `void`).
3. It is **called automatically by the compiler** — you do not call it explicitly.